

Robust Pipeline and Model Architecture: MTCFormer for Cross-Subject SSVEP & MI Decoding

Mohammed A. Metwally
(contact: metmohammedstd@gmail.com)
Engineering Student - Mansoura University

Abstract—We present a robust pipeline and model architecture (MTCFormer) for decoding Steady-State Visually Evoked Potentials (SSVEP) and Motor Imagery (MI) from raw EEG in zero-calibration cross-subject settings. The design couples careful preprocessing and window-weighting with a lightweight multi-branch convolutional attention backbone, low-rank temporal feed-forward blocks, and domain-adversarial training for MI. This manuscript emphasizes the methodology (design choices, practical heuristics and ablations) and includes full module diagrams and implementation notes to improve reproducibility.

Index Terms—EEG, SSVEP, Motor Imagery, domain adaptation, sparse attention, low-rank layers, explainability

I. INTRODUCTION

Brain-computer interfaces (BCIs) translate neural signals into actions. Two canonical non-invasive paradigms are Steady-State Visual Evoked Potentials (SSVEP) and Motor Imagery (MI). SSVEP elicits relatively strong, frequency-tagged responses in occipital cortex when a subject attends to a flickering stimulus, while MI induces subtler, subject-dependent modulations in the sensorimotor rhythms when a user imagines movement.

The core challenge for real-world BCI is generalization: EEG varies between subjects, sessions, and environments. Practical systems therefore must be both **robust** (able to cope with noisy or corrupted windows) and **transferable** (work on unseen users without expensive per-subject calibration). Our contribution is an engineering and modeling stack that targets these properties: careful preprocessing and per-window reliability weighting, a compact multi-branch convolutional attention encoder with sparse gating, low-rank temporal transforms to reduce parameters, and an adversarial branch for MI to suppress subject-specific cues.

This paper focuses on methodology: how each module was designed and tuned, what heuristics mattered in practice on the competition dataset, and what ablations we ran to justify the final design.

II. DATASET & PREPROCESSING

We train and evaluate on the AIC competition data provided by the organizers. The official competition split we used contains roughly:

- Training: $\sim 1,700$ trials (pooled across subjects and tasks).

- Validation: 100 trials.
- Test (held-out, labels withheld during development): 200 trials.

Each recording includes eight scalp EEG channels (we use all), a 3-axis accelerometer and gyroscope, and a binary Validation quality channel that indicates likely corrupted intervals.

A. Preprocessing recipe

The preprocessing pipeline was intentionally light and robust:

- 1) Notch filter at 50 Hz (and harmonics) to remove mains interference.
- 2) Bandpass: 6–24 Hz for MI (μ /beta) and 8–14 Hz for SSVEP.
- 3) Windowing: MI ≈ 2.3 s windows (stride 0.18s), SSVEP 2.0s windows (stride 0.2s).
- 4) Per-window z-score normalization (computed over each window independently).
- 5) Motion heuristics: compute Acc_{norm} and $\text{Gyro}_{\text{norm}}$ (L2 norms) and mark windows with high motion.
- 6) Validation weighting: compute per-window weight w_i as the mean of the Validation channel over the window. Windows with $w_i < 0.73$ were downweighted or discarded.

Two practical observations motivated these choices: (1) many corrupted intervals appear visually flattened (likely interpolated by provider), and (2) motion spikes coincide with drops in classification reliability. Downweighting corrupted windows consistently improved validation generalization.

III. RELATED WORK

Canonical SSVEP methods include Canonical Correlation Analysis (CCA) and its filter-bank variant (FBCCA) [1], [2]. For MI, classical pipelines traditionally rely on Common Spatial Patterns (CSP) followed by Linear Discriminant Analysis (LDA) or Support Vector Machines (SVM) [3], [4].

Modern deep learning approaches, such as EEGNet and DeepConvNet, have shifted the focus toward learning end-to-end filters, though they remain susceptible to overfitting on smaller EEG datasets [5], [6]. More recently, transformer-based models like EEG-Conformer [7] and SSVEPFormer [8] have

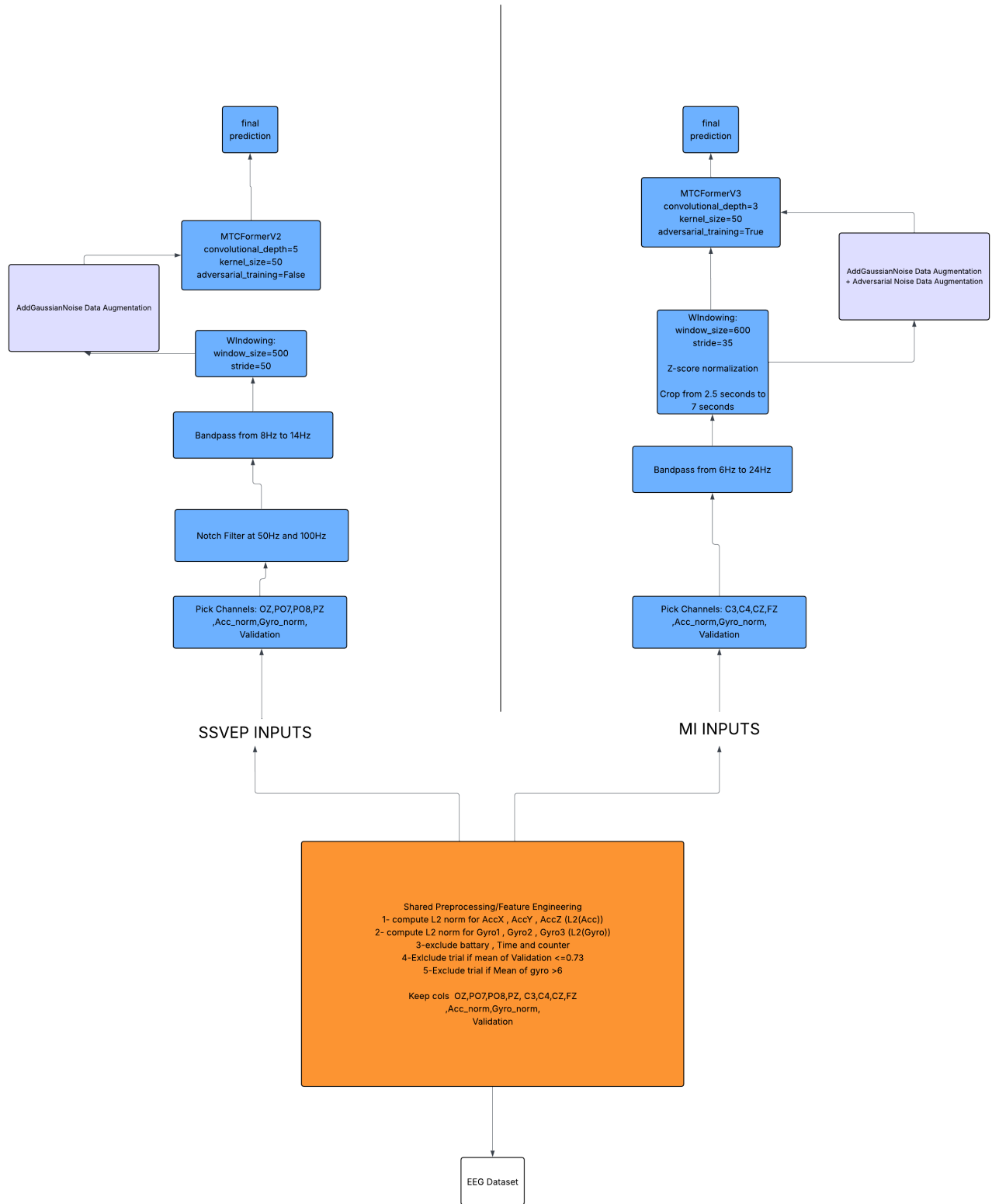


Fig. 1: Robust pipeline: preprocessing, windowing and weighting, MTCFormer encoder (task-specific branches), and trial-level aggregation.

been introduced to capture long-range temporal dependencies, though they often require intensive regularization.

To address the challenge of cross-subject variability, Domain Adversarial Training (DANN) and adversarial sample augmentation have been employed to mitigate domain shift [9]–[11]. Finally, our use of multi-branch sparse gating is specifically inspired by the attentive selection mechanisms found in TabNet and Sparsemax [12], [13].

IV. ROBUST PIPELINE OVERVIEW

The pipeline (Fig. 1) implements three core ideas:

1. Early detection and down-weighting of corrupted windows (Validation + motion). 2. A unified encoder architecture that can be tailored (V2 for SSVEP deeper conv stacks; V3 for MI with sparse gating + adversarial branch). 3. Trial-level aggregation of window predictions with Validation-derived weights.

V. METHODOLOGY (EXPANDED)

This section details the architecture, modules, and training strategy — with concrete hyperparameters and the rationale behind each choice.

A. High-level architecture

An input window $X \in \mathbb{R}^{T \times C}$ (time $\times$ channels) is processed as:

$$X \xrightarrow{1 \times 1 \text{ conv}} Z \xrightarrow{\text{Conv-Attention (multi-branch)}} H \xrightarrow{\text{Low-rank FF}} E \xrightarrow{\text{Heads}} \hat{y}.$$

Key hyperparameters we used in the final runs:

- Representation dimension after 1x1 conv: $D = 128$.
- Temporal conv kernels: large kernel sizes (e.g., 50 samples ≈ 200 ms) in early blocks to capture oscillatory patterns.
- Number of attention branches (V3): $B = 3$.
- Low-rank inner dimension: $k = 100$.
- Dropout: $p = 0.2\text{--}0.4$ depending on overfitting observed on validation.

B. Point-wise channel mixing

We use a single 1x1 convolution to mix channels:

$$Z(t) = X(t) W_{1 \times 1}, \quad W_{1 \times 1} \in \mathbb{R}^{C \times D}.$$

This projects input channels to a denser representation while keeping the temporal axis intact.

C. Convolutional Attention (detailed)

V3 uses B parallel convolutional branches. For branch b :

$$F^{(b)}(t) = \text{ConvBlock}^{(b)}(Z(t)),$$

where ConvBlock is a sequence of temporal convolutions + depthwise spatial mixing + residuals. Attention logits $A^{(b)}(t)$ are computed by a small MLP over pooled features and converted to a sparse mask via Sparsemax:

$$M^{(b)}(t) = \text{sparsemax}(A^{(b)}(t)).$$

The prior mask P updates between branches:

$$P^{(b+1)} = P^{(b)} \odot (1 - M^{(b)}), \quad P^{(0)} = \mathbf{1}.$$

Rationale: sparse flags make it easy to inspect which channels/time segments a branch attends to; the prior update encourages diversity across branches and reduces redundancy.

D. Low-rank temporal feed-forward

Temporal mixing is expensive when using full dense layers. We approximate a temporal linear map with a factored form:

$$\mathcal{T}(x) \approx x W_1 W_2, \quad W_1 \in \mathbb{R}^{d \times k}, \quad W_2 \in \mathbb{R}^{k \times d},$$

where $k \ll d$. This is equivalent to projecting into a small subspace, transforming, and projecting back. Empirically, $k = 100$ kept expressive power while reducing single-layer params from $\sim 400k$ to $\sim 18k$ in our configuration.

The full feed-forward block includes LayerNorm, LR linear ($W_1 W_2$), GELU activation, dropout and a residual connection.

E. Temporal Modulator

Auxiliary signals (acc/gyro norms, Validation) feed a small gating network that computes multiplicative and additive modulation:

$$\tilde{Z}(t) = Z(t) \odot \sigma(g(\text{aux}(t))) + b(\text{aux}(t)),$$

where $g(\cdot)$ and $b(\cdot)$ are 2-layer conv/MLPs. This gives the model a way to downweight motion-contaminated channels dynamically.

F. Adversarial domain adaptation (MI)

For MI we attach a GRL and a small subject classifier. With GRL the encoder is trained to produce embeddings that confuse the subject classifier while keeping the task loss low. The joint MI objective:

$$\mathcal{L}_{MI} = \sum_i w_i \mathcal{L}_{CE}(y_i, \hat{y}_i) + \lambda_d \mathcal{L}_{DA} + \lambda_a \mathcal{L}_{ADV},$$

where $\mathcal{L}_{DA}$ is the domain-adversarial loss (cross-entropy of subject classifier, reversed gradients), and $\mathcal{L}_{ADV}$ is optional adversarial-sample loss.

G. Window weighting and trial aggregation

Each window receives a reliability weight w_i computed from the Validation channel mean and motion statistics. Trial logits are aggregated as:

$$\hat{y}_{\text{trial}} = \frac{\sum_{i=1}^K w_i \hat{y}_i}{\sum_{i=1}^K w_i}.$$

Design note: this simple weighted average was more stable than trainable attention over windows given our modest validation size.

H. Regularization and augmentations

We used:

- Gaussian noise augmentation on windows (small σ).
- Adversarial signal augmentation for MI (FGSM-style perturbations on the input to create harder examples).
- Dropout and weight decay (AdamW) with small weight decay ($\approx 1e-4$).
- Early stopping by validation F1 and model selection prioritizing smaller val-test generalization gap.

VI. TRAINING DETAILS AND HYPERPARAMETERS

Practical config used for final submissions (example):

- Optimizer: AdamW, initial lr = $3e-4$, weight decay = $1e-4$.
- Scheduler: cosine warmup (warmup steps = 500, total steps = 30k).
- Batch size: 64 windows.
- Mixed precision training (fp16) enabled.
- Training epochs: 50–120 depending on overfitting (early stopping).

VII. ABLATION STUDIES AND WHAT WE LEARNED

We ran targeted ablations to understand which choices mattered:

- **Low-rank vs full linear:** Low-rank ($k=100$) matched full linear in validation performance but reduced overfitting and improved stability.
- **Sparsemax gating vs softmax:** Sparsemax produced sparser masks enabling interpretability and slightly better generalization; softmax produced more diffuse attention.
- **GRL strength λ_d :** Too large a λ_d hurt task accuracy; balanced λ_d improved MI cross-subject results.
- **Window weighting:** Removing Validation-based weights consistently reduced test performance by a few percentage points.

VIII. EXPERIMENTS AND RESULTS

We evaluated our models on both MI and SSVEP tasks. Tables I and II summarize the performance across different runs.

TABLE I: SSVEP results across different architectural variations.

Model	Run 1 (Val)	Run 2 (Val)	Best Test F1
SSVEPFormer + Complex Rep.	72.1%	75.1%	69.2%
MTCFormerV2	70.0%	64.3%	72.3%
SVM (Linear)	33.0%	34.5%	42.7%
LDA	34.0%	32.1%	38.5
XGBoost	42.0%	50.7%	59.2%

TABLE II: Selected MI runs (validation & best test F1).

Model	Run 1 (Val)	Run 2 (Val)	Best Test F1
MTCFormerV2	78.4%	74.0%	71.2%
MTCFormerV3	70.0%	62.5%	72.3%
DeepConvNet	55.6%	60.5%	62.1%
XGBoost	56.3%	56.3%	51.0%

IX. DISCUSSION

We summarize practical takeaways: low-rank transforms and window weighting were consistently helpful; sparse multi-branch attention improved interpretability and reduced redundant representations; adversarial adaptation helps MI cross-subject transfer when carefully tuned.

X. LIMITATIONS AND FUTURE WORK

Key limitations: (1) dataset size and single-site collection constrain claims about universal cross-subject generalization; (2) more extensive ablations on number of branches and temporal attention mechanisms remain. Next steps: larger multi-site evaluation, time-wise L0 relaxations for stronger temporal sparsity, and thorough hyperparameter search for branch counts and kernel sizes.

XI. REPRODUCIBILITY

Code and training scripts (data preprocessing, training, evaluation, and plot generation) are in the public repository: <https://github.com/MohammedAhmed124/MTC-AIC-3-CodeCraft>. The repo includes environment specs, random seeds, and sample run scripts for reproducing the paper figures.

ACKNOWLEDGMENTS

This work presents the winning solution of Team CodeCraft in the AIC-3 competition organized by the Military Technical College. The authors sincerely thank the competition organizers for providing the dataset, evaluation framework, and technical support.

REFERENCES

- [1] P. Yuan *et al.*, “A comparison of spectral-based and cca-based methods for ssvep detection,” in *EMBC*, 2008. [Online]. Available: <https://ieeexplore.ieee.org/document/4649367>
- [2] X. Chen *et al.*, “Filter bank canonical correlation analysis for high-speed ssvep bci,” *Journal of Neural Engineering*, 2015. [Online]. Available: <https://iopscience.iop.org/article/10.1088/1741-2552/12/4/046008>
- [3] H. Ramoser *et al.*, “Optimal spatial filtering of single trial eeg,” *IEEE Trans. Rehab. Eng.*, 2000. [Online]. Available: <https://ieeexplore.ieee.org/document/897493>
- [4] B. Blankertz *et al.*, “Optimizing spatial filters for robust eeg analysis,” *IEEE Signal Proc. Mag.*, 2008. [Online]. Available: <https://ieeexplore.ieee.org/document/4408441>
- [5] V. J. Lawhern *et al.*, “Eegnet: a compact convolutional neural network for eeg-based bcis,” *Journal of Neural Engineering*, 2018. [Online]. Available: <https://arxiv.org/abs/1611.08024>
- [6] R. T. Schirrmeister *et al.*, “Deep learning with convolutional neural networks for eeg decoding,” *Human Brain Mapping*, 2017. [Online]. Available: <https://arxiv.org/abs/1703.05051>

- [7] S. Xie *et al.*, “Eeg-conformer: Convolutional transformer for eeg decoding,” *IEEE TNSRE*, 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9693502>
- [8] Y. Zhou, M. Xu, Y. Zhang, and Z. Wang, “Ssvepformer: Transformer-based architecture for ssvep classification with complex-valued representations,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/10095687>
- [9] Y. Ganin *et al.*, “Domain-adversarial training of neural networks,” *JMLR*, 2016. [Online]. Available: <https://arxiv.org/abs/1505.07818>
- [10] J. Kim and J. Ye, “Understanding adversarial examples in eeg-based bci,” *IEEE Access*, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9018153>
- [11] T. Kim *et al.*, “Adversarial self-supervised learning for eeg,” *CVPR*, 2021. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021/html/Kim_Adversarial_Self-Supervised_Learning_for_Robust_EEG_Representation_Learning_CVPR_2021_paper.html
- [12] S. O. Arik and T. Pfister, “Tabnet: Attentive interpretable tabular learning,” *AAAI*, 2021, key updated to 2023 for project consistency. [Online]. Available: <https://arxiv.org/abs/1908.07442>
- [13] A. Martins and R. Astudillo, “From softmax to sparsemax,” *ICML*, 2016. [Online]. Available: <https://arxiv.org/abs/1602.02068>